Analysis of massive data sets

http://www.fer.hr/predmet/avsp

Izv. prof. dr. sc. Marin Šilić

Izv. prof. dr. sc. Goran Delač

Izv. prof. dr. sc. Klemo Vladimir

Doc. dr. sc. Adrian Satja Kurdija

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Analysis of Massive Data Sets: Mining Large Graphs – Link Analysis

Marin Šilić

Overview

- Motivation
- Link Analysis
- Flow Formulation
- Matrix Formulation
- Power Iteration Method
- Google Formulation
- Implementation Details

Copyright © 2026 CCL: Analysis of massive data sets

Motivation

- **Data Naturally Represented as Graph**
  - Social Networks

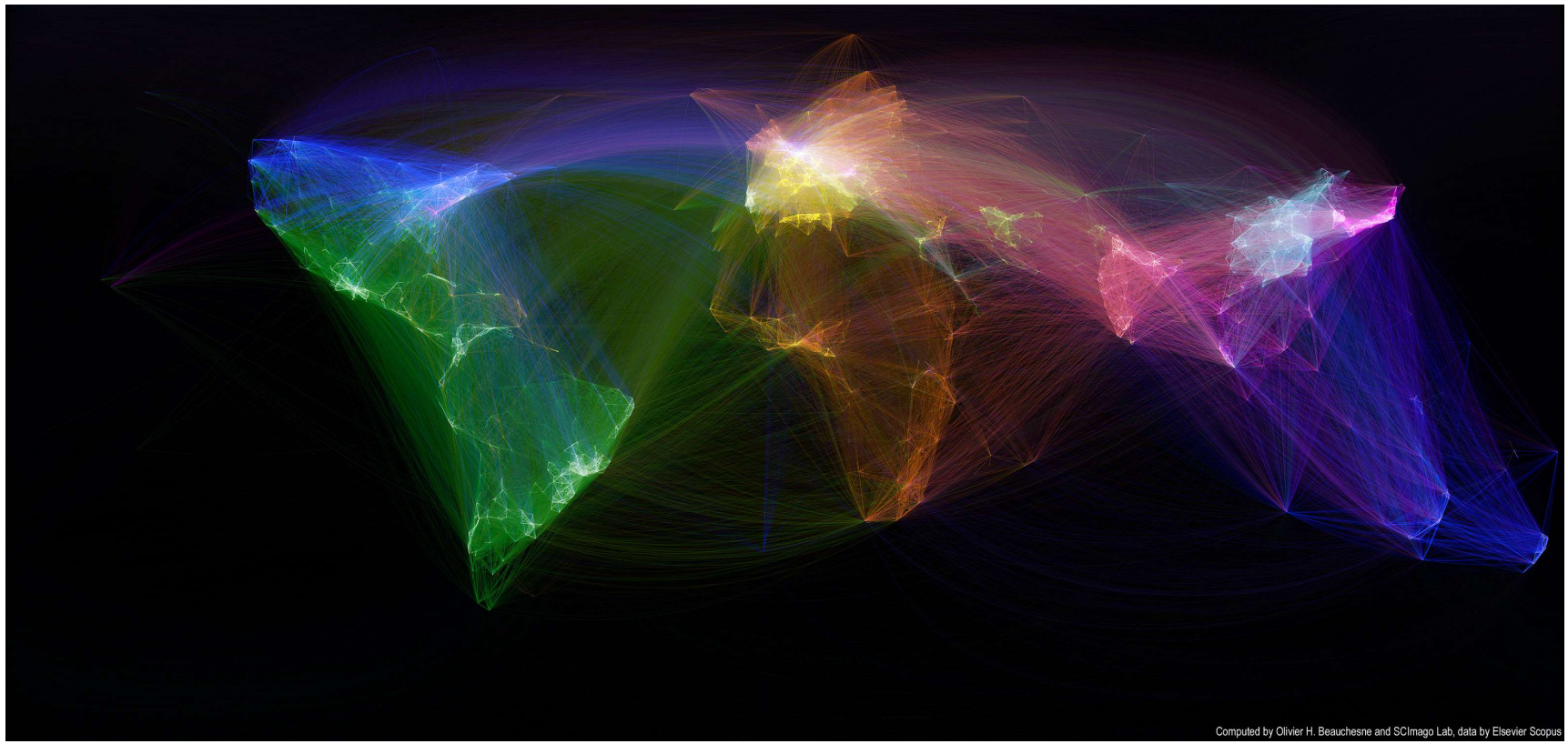


Facebook's Social Network Graph: Paul Butler, intern at Facebook, Visualization of the locality of friendship, 2010

Copyright © 2026 CCL: Analysis of massive data sets

Motivation

- **Data Naturally Represented as Graph**
  - Map of scientific collaboration

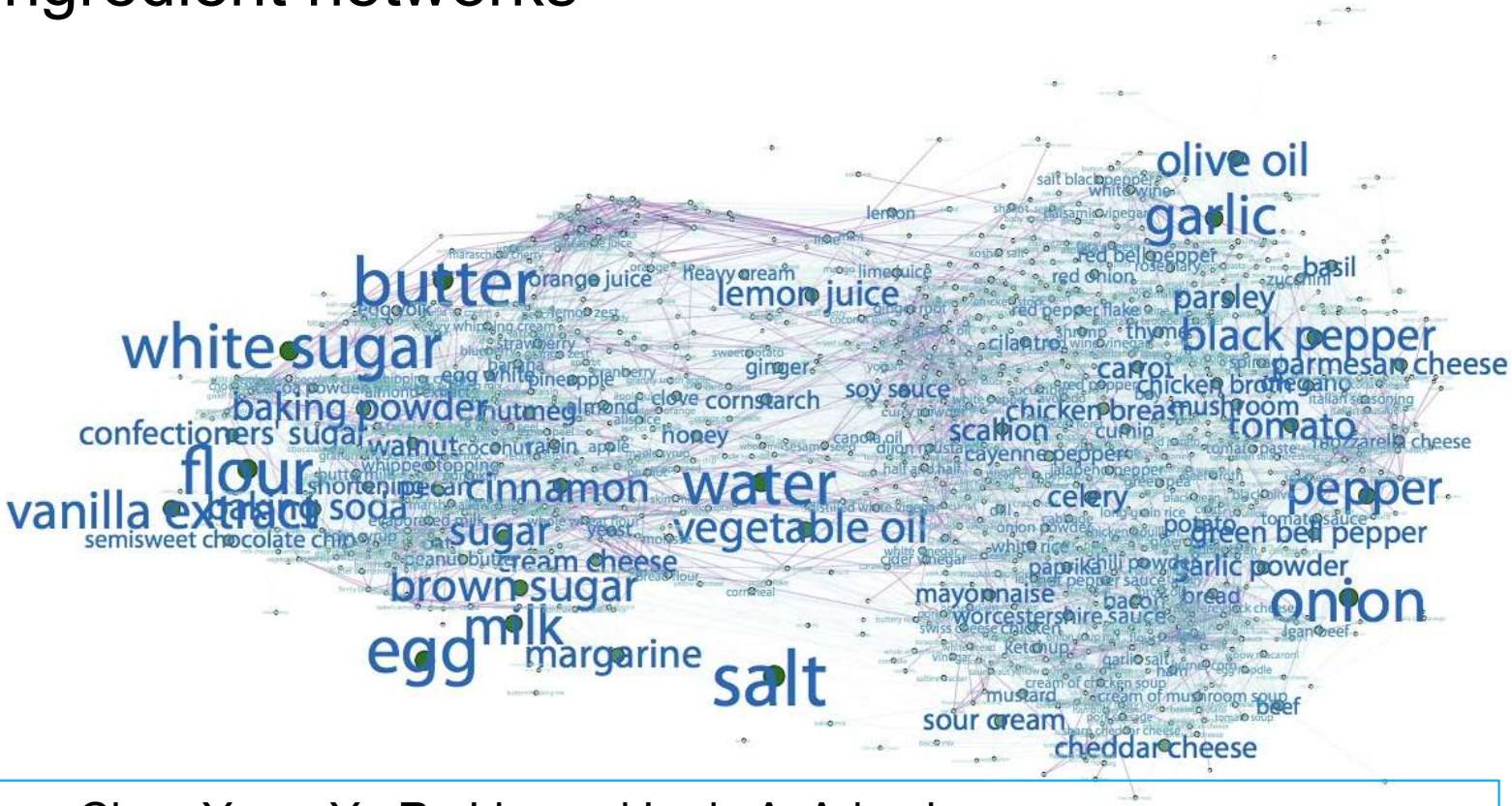


http://olihb.com/2011/01/23/map-of-scientific-collaboration-between-researchers/

Copyright © 2026 CCL: Analysis of massive data sets

Motivation

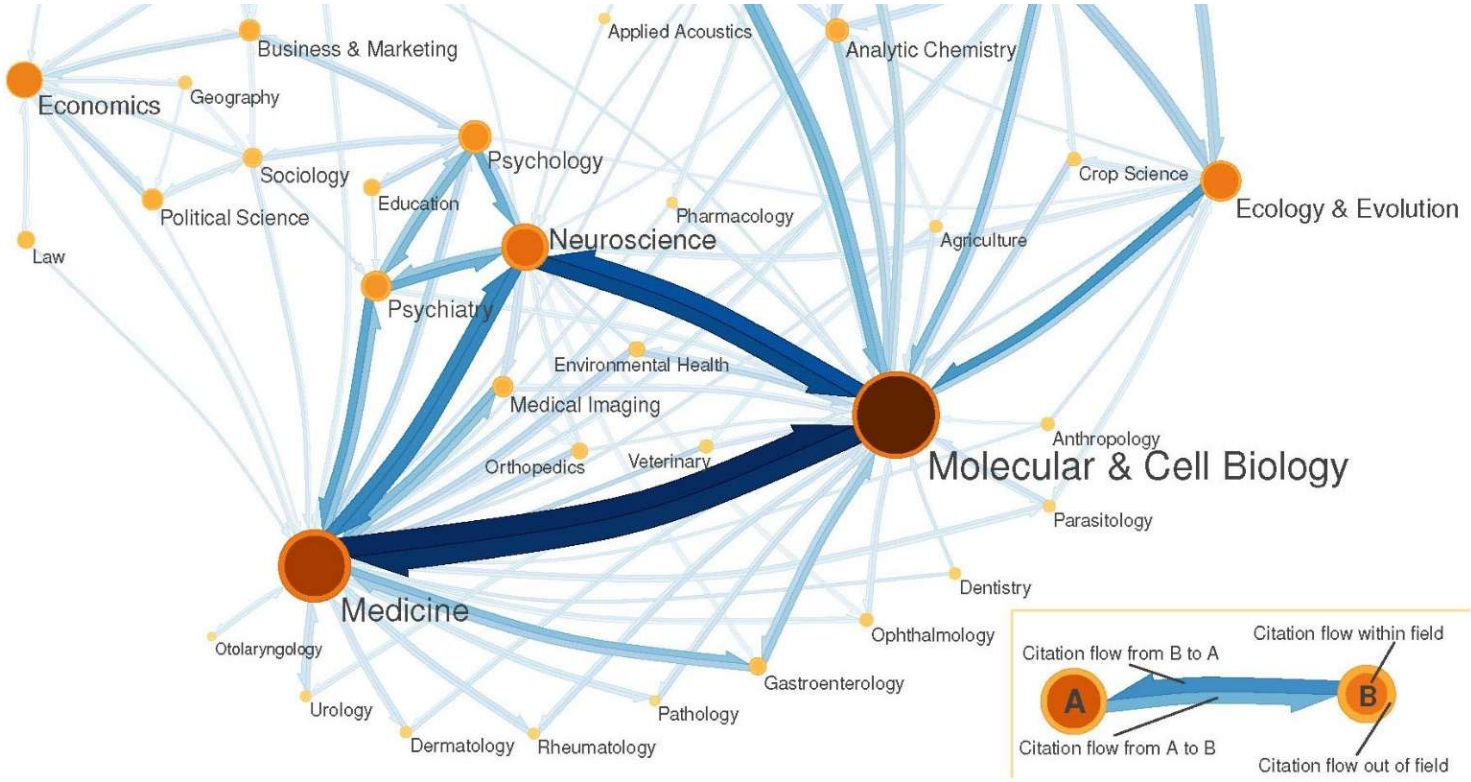
- **Data Naturally Represented as Graph**
  - Ingredient networks



Teng, Chun-Yuen, Yu-Ru Lin, and Lada A. Adamic.
"Recipe recommendation using ingredient networks."
Proceedings of the 4th Annual ACM Web Science Conference. ACM, 2012.

Motivation

- **Data Naturally Represented as Graph**
  - The science citation map

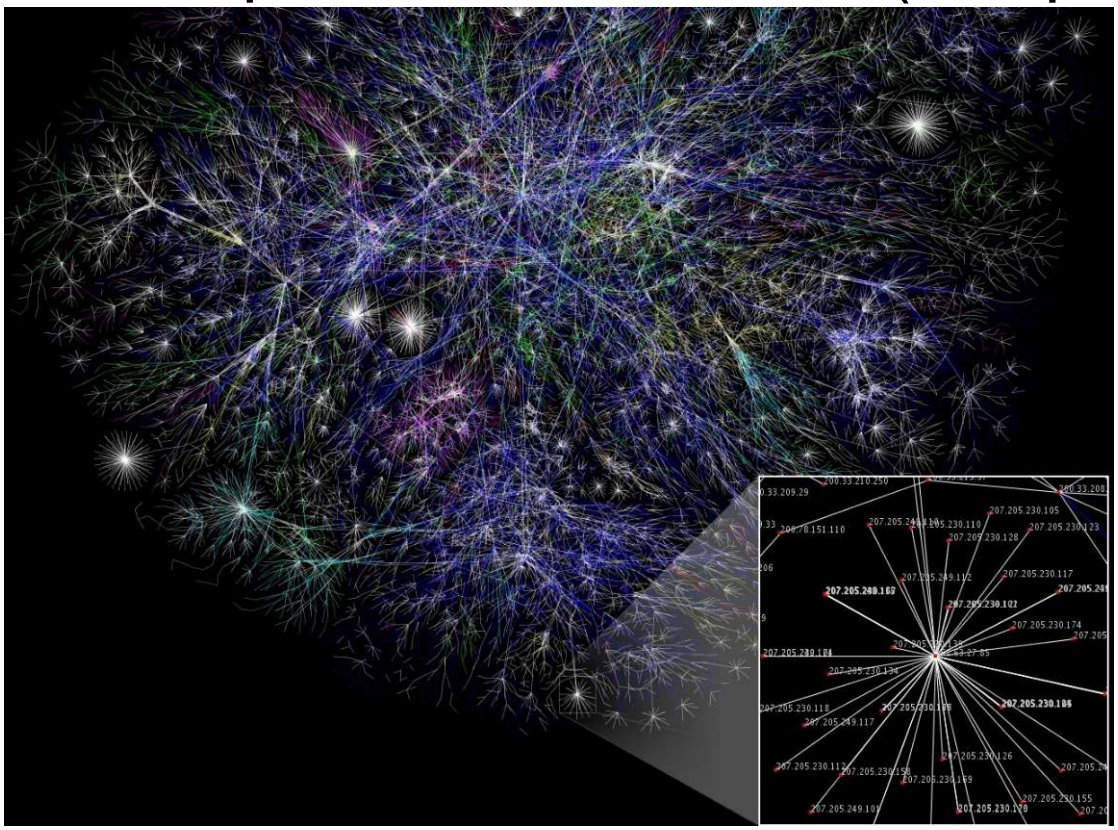


West, Jevin D. *Eigenfactor: ranking and mapping scientific knowledge*. Diss. University of Washington, 2010.

Copyright © 2026 CCL: Analysis of massive data sets

Motivation

- **Data Naturally Represented as Graph**
  - The map of Internet in 2005 (Wikipedia)

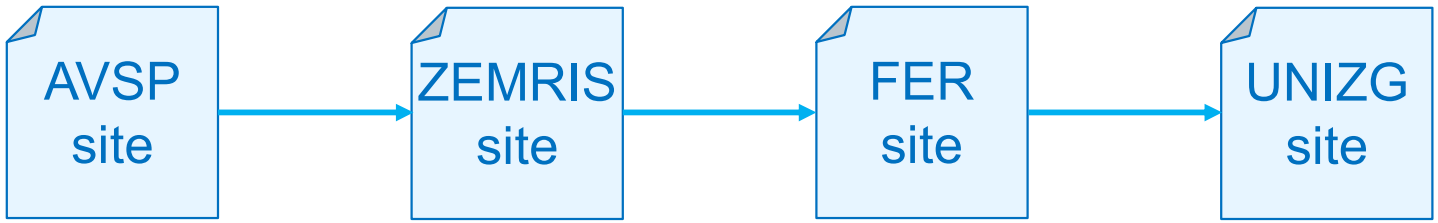


Partial map of the Internet based on the January 15, 2005 data found on opte.org. Each line is drawn between two nodes, representing two IP addresses.

Copyright © 2026 CCL: Analysis of massive data sets

Link Analysis

- **Imagine the Web as a Graph**
  - Directed graph
    - Nodes: Websites
    - Edges: Link among websites – Hyperlinks



Copyright © 2026 CCL: Analysis of massive data sets

Link Analysis

- **How to organize the Web?**
  - First try: Human organized
    - Web directories
      - Yahoo, DMOZ, LookSmart
    - Put each new website in its category
    - Issue: Web is huge, the approach does not scale!!
  - Second try: Web Search
    - Information Retrieval
      - Find relevant docs in a small and trusted subset
      - Newspapers articles and similar trusted data corpus
    - Issue: Again, web is huge, plenty of untrusted documents, random things, web spam, etc.



Link Analysis

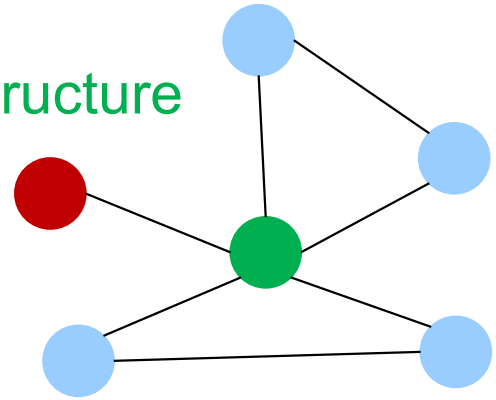
- **Two main challenges of Web Search**
  - There are plenty of information sources on the web
    Who do we trust?
    - Idea: Use structure of the web graphs
    - Trusted pages are likely to point to each other!
  - What is the best (appropriate) answer to query "media"
    - There is an ambiguity in this query, no single right answer
    - Idea: Again, utilize structure of the web
    - The pages that actually know about media might all be pointing to many media sites

Copyright © 2026 CCL: Analysis of massive data sets

Link Analysis

- **Ranking Nodes on the Graph**
  - All web pages are not equally important
    - www.some-unknown-page.com
    - www.fer.hr
  - The structure of the web graph tells something
    - There is a large diversity when considering node connectivity
    - The idea: Lets rank pages by the link structure

The green node is more important than the red node



Link Analysis

- **Link Analysis Algorithms**
  - Computing the importance of the nodes in the graph
  - Various solutions to the Link Analysis problem
    - Page Rank
    - Topic-Specific (Personalized) Page Rank
    - Web Spam Detection Algorithms

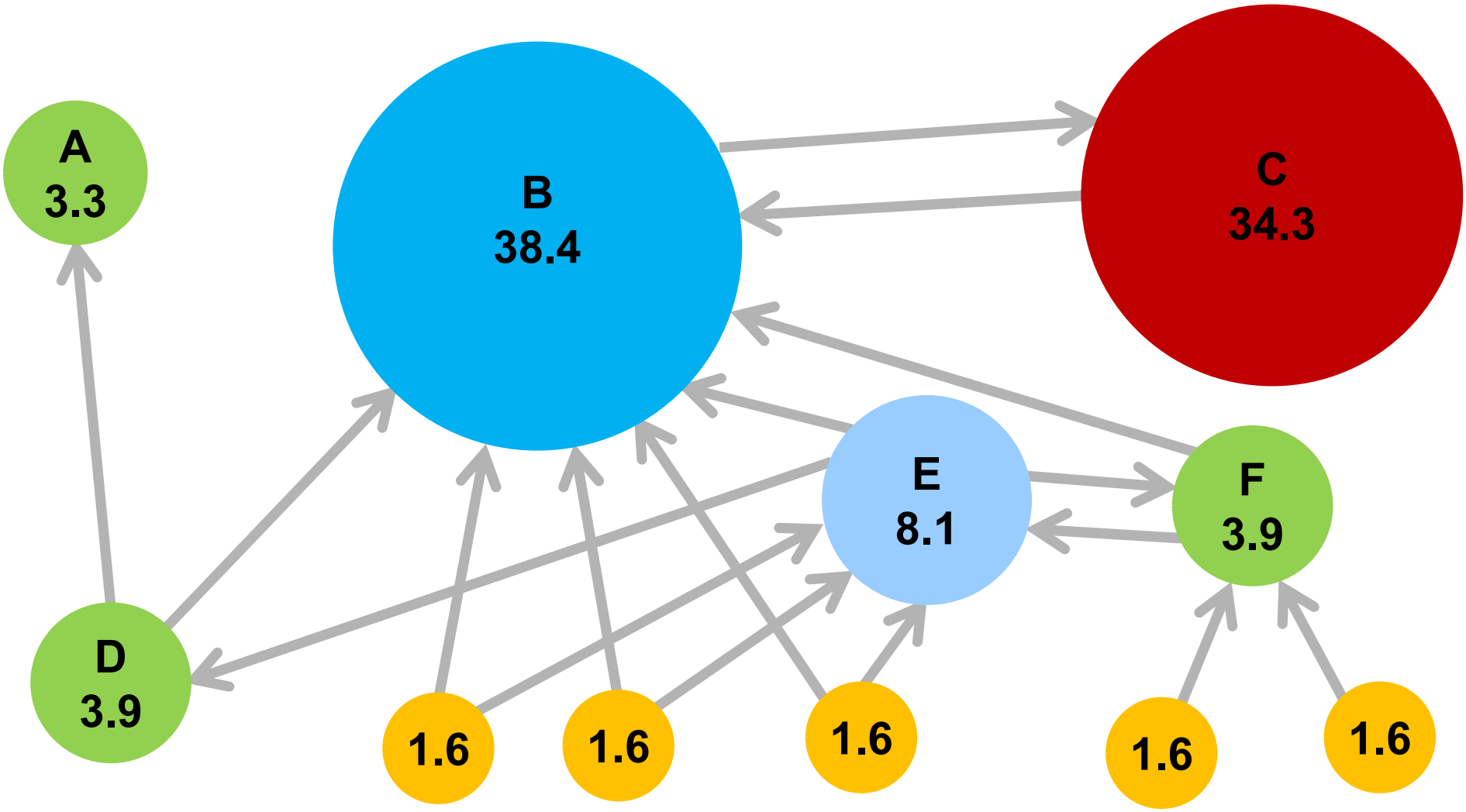
Flow Formulation

- **Count Node's Links**
  - Idea: Utilize graph structure
    Count node links as votes
    - Incoming links vs Outgoing links
  - Imagine incoming links as votes
    - www.fer.hr has 24622 incoming links
    - www.some-unknown-page.com has 0 incoming links
  - However, all incoming links are not equally important
    - Links originating from important pages are worth more
    - The nature of the problem is recursive

Copyright © 2026 CCL: Analysis of massive data sets

Flow Formulation

- Graph Example: PageRank Scores

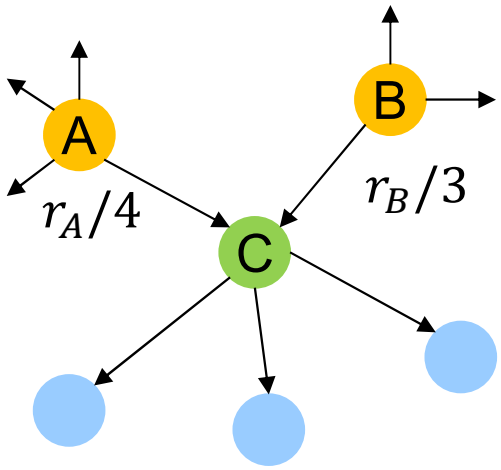


Copyright © 2026 CCL: Analysis of massive data sets

Flow Formulation

- **Page Rank: Recursive Formulation**
  - Idea: Incoming link's value is proportional to the importance of its source node
  - If node j with importance r_j has n outgoing links
    - Each links gets r_j/n votes
  - Node j's importance is equal to the sum of the votes of its incoming links

$$r_C = \frac{r_A}{4} + \frac{r_B}{3}$$

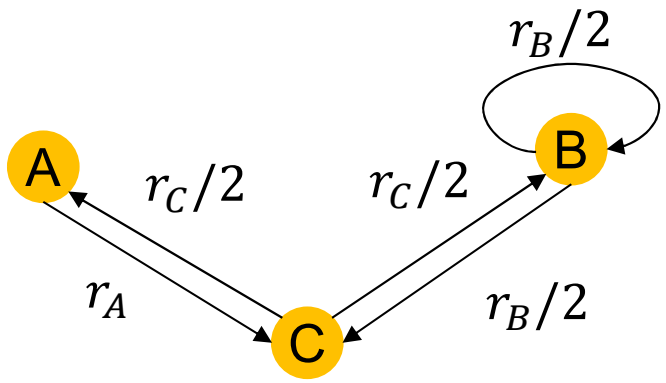


Flow Formulation

- **Page Rank: The "Flow" Model**
  - A link (vote) from an important page is worth more
  - A node is important if it is linked by other important nodes
  - Rank definition r_j for node j:

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}$$

d_i is the out-degree of the node i



"Flow" equations:

$$r_A = \frac{r_C}{2}$$

$$r_B = \frac{r_B}{2} + \frac{r_C}{2}$$

$$r_C = r_A + \frac{r_B}{2}$$

Copyright © 2026 CCL: Analysis of massive data sets

Flow Formulation

- **Page Rank: The "Flow" Model**
  - 3 equations, 3 unknowns, no constants
    - There is no unique solution
  - However, there is an additional constraint
    - We force the uniqueness
    - The sum of all node ranks sums to 1
  - Unique solution $r_A = \frac{1}{5}, r_B = \frac{2}{5}, r_C = \frac{2}{5}$
  - Gauss elimination method works for small graphs
    - $O(N^3)$, it does not scale already for $N = 10^4$
    - We need some other solution

"Flow" equations:

$$r_A = \frac{r_C}{2}$$
$$r_B = \frac{r_B}{2} + \frac{r_C}{2}$$
$$r_C = r_A + \frac{r_B}{2}$$

Copyright © 2026 CCL: Analysis of massive data sets

Matrix Formulation

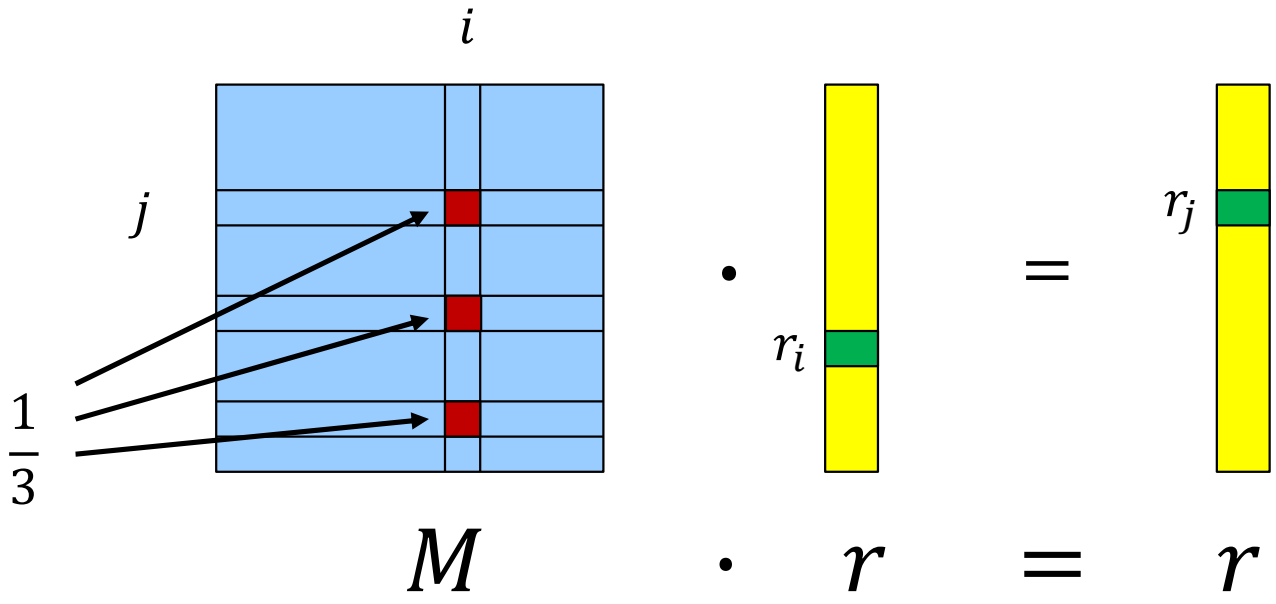
- **Write Flow Formulation in Matrix M**
  - Stochastic Adjacency Matrix M
  - If $i \to j$, then $M_{ji} = 1/d_i$, else $M_{ji} = 0$
  - M is a column stochastic matrix
    - Columns of the matrix sum to 1
  - Rank vector r: vector with an entry per page
    - r_i is the importance score of node i
    - $\sum_i r_i = 1$
  - The flow equation can be written as: $r_j = \sum_{i \to j} \frac{r_i}{d_i}$

$$r = M \cdot r$$

Copyright © 2026 CCL: Analysis of massive data sets

Matrix Formulation

- **Example**
  - Flow equation $r_j = \sum_{i \to j} \frac{r_i}{d_i}$
  - Matrix formulation $M \cdot r = r$
    - Page i links to 3 pages, including j



Copyright © 2026 CCL: Analysis of massive data sets

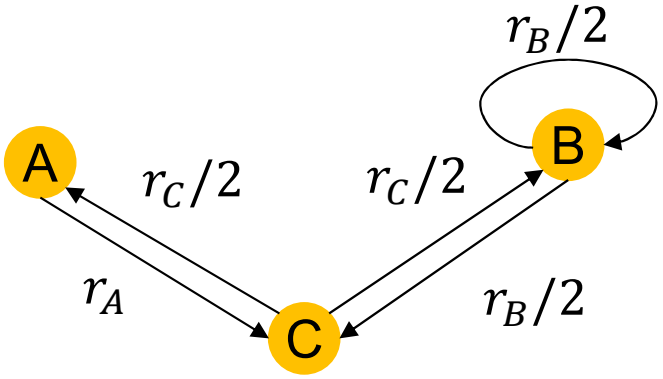
Matrix Formulation

- **Flow Equation:** $M \cdot r = r$
- **Eigenvector:** $A \cdot v = \lambda \cdot v$
  - v is an eigenvector of matrix A, with the corresponding eigenvalue λ
  - Hence, the rank vector r is an eigenvector of the stochastic matrix M
    - Its corresponding eigenvalue is 1
    - The largest (dominant) eigenvalue of M is 1
      - M is column stochastic (each column sums to 1, non-zero entries)
      - r is unit length (non-zero entries sum to 1)
- **Power Iteration Method**
  - Algorithm for finding the dominant eigenvector

Copyright © 2026 CCL: Analysis of massive data sets

Matrix Formulation

- **Back to our Example…**



"Flow" equations:

$$r_A = \frac{r_C}{2}$$

$$r_B = \frac{r_B}{2} + \frac{r_C}{2}$$

$$r_C = r_A + \frac{r_B}{2}$$

	A	B	C
A	0	0	1/2
B	0	1/2	1/2
C	1	1/2	0

M

$$\begin{pmatrix} r_A \\ r_B \\ r_C \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1/2 \\ 0 & 1/2 & 1/2 \\ 1 & 1/2 & 0 \end{pmatrix} \cdot \begin{pmatrix} r_A \\ r_B \\ r_C \end{pmatrix}$$

$r = M \cdot r$

Power Iteration Method

- **Input: Graph with N nodes**
- **Power Iteration: iterative algorithm**
  - There are N nodes
  - Initialize: $r^{(0)} = \left[\frac{1}{N}, \ldots, \frac{1}{N}\right]^T$
  - Iteratively compute: $r^{(t+1)} = M \cdot r^{(t)}$
  - Until: $|r^{(t+1)} - r^{(t)}|_1 > \epsilon$
    - $|x|_1 = \sum_{1 \le i \le N} |x_i|$, L_1 norm
    - Some other vector norm can be used, for instance Euclidean

Python code example

```
import numpy as np

M = np.matrix('0 0 0.5; 0 0.5 0.5; 1 0.5 0')
r_p = np.matrix('0.35; 0.30; 0.35')

r_n = M * r_p
i = 1

while np.linalg.norm(r_n - r_p) > 0.01:
        i += 1
        r_p = r_n
        r_n = M * r_p

print(i)
print(r_n)
```

Copyright © 2026 CCL: Analysis of massive data sets

Power Iteration Method

Power Iteration Example

- Algorithm

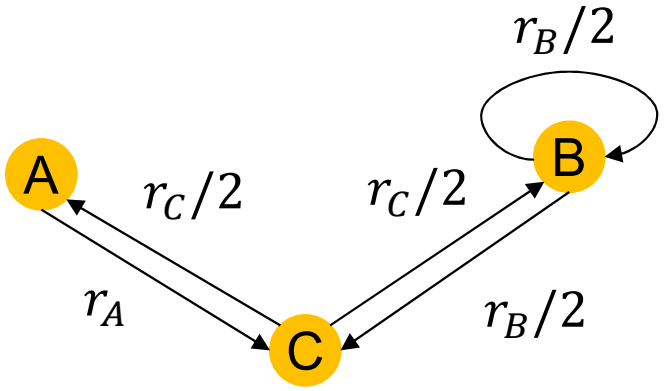
for $j = 1$ **to** N

$r_j := 1/N$

do

$r_j' := \sum_{i \to j} \frac{r_i}{d_i}$

while $(r_j' \neq r_j)$



"Flow" equations:

$$r_A = \frac{r_C}{2}$$

$$r_B = \frac{r_B}{2} + \frac{r_C}{2}$$

$$r_C = r_A + \frac{r_B}{2}$$

- Results:

$$\begin{bmatrix} r_A \\ r_B \\ r_C \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \begin{bmatrix} 1/6 \\ 2/6 \\ 3/6 \end{bmatrix} \begin{bmatrix} 3/12 \\ 5/12 \\ 4/12 \end{bmatrix} \dots \begin{bmatrix} 3/15 \\ 6/15 \\ 6/15 \end{bmatrix}$$

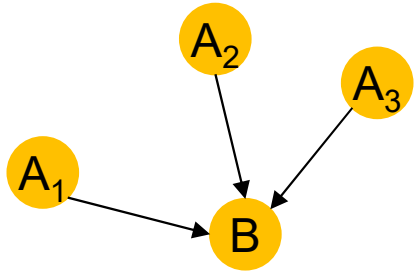
Iteration 0 1 2 …

Copyright © 2026 CCL: Analysis of massive data sets

Power Iteration Method

- **Random Walk Interpretation**
  - Imagine a random walker that explores a graph
    - At some time t, the walker is at some node A
    - At time $t+1$, the walker randomly follows some outgoing link from node A
    - The walker gets to some node B linked from node A
    - The random walk process repeats indefinitely

$$r_B = \sum_{A \to B} \frac{r_A}{d_A}$$



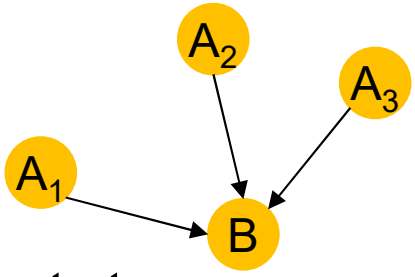
  - We define:
    - $p(t)$ as the vector whose i^{th} coordinates corresponds to the probability that the walker is at node i at time t
    - $p(t)$ is a probability distribution over nodes

Copyright © 2026 CCL: Analysis of massive data sets

Power Iteration Method

Random Walk Interpretation

- Having a graph structure and $p(t)$, where is the walker at time $p(t+1)$?
  - Follows the links uniformly at random
  - $p(t+1) = M \cdot p(t)$



- If at some point the walker reaches a state,
  - $p(t+1) = M \cdot p(t) = p(t)$,
    then $p(t)$ is a stationary distribution of random walk
- Rank vector r satisfies $r = M \cdot r$
  - Hence, r is stationary distribution for the random walk

Copyright © 2026 CCL: Analysis of massive data sets

Power Iteration Method

- **Theory of Random Walks**
  - Markov processes
    - Processes that are memoryless
    - One can make predictions for the future based solely on its present state
  - If the initial graph satisfies certain conditions, the stationary distribution is unique, and it will be always reached
  - Thus, regardless what the initial distribution at time $t = 0$ is, the stationary distribution will be achieved

Copyright © 2026 CCL: Analysis of massive data sets

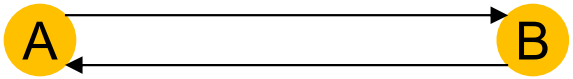
Google Formulation

- **So far:**
  - Flow equation: $r_j = \sum_{i \to j} \frac{r_i}{d_i}$
  - Matrix formulation: $r = M \cdot r$
  - Two questions:
    1) Does it converge?
    2) Does the solution make sense?

Copyright © 2026 CCL: Analysis of massive data sets

Google Formulation

- **Does it converge?**



$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{d_i}$$

  - Solution

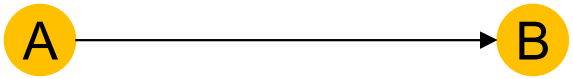
    - r_A = 1 0 1 …
    - r_B = 0 1 0 …

    Iteration 0 1 2 …

It will never converge!

Google Formulation

- **Does the solution makes sense?**
  - Solution



$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{d_i}$$

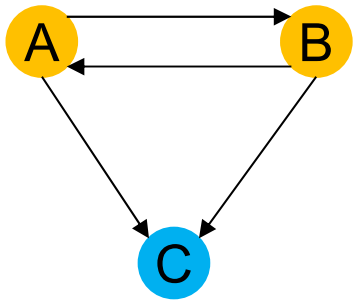
r_A =	1	0	0	…
r_B =	0	1	0	…
Iteration	0	1	2	…

It will converge to a meaningless result!

Copyright © 2026 CCL: Analysis of massive data sets

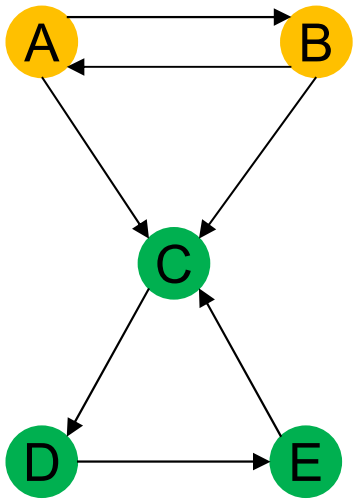
Google Formulation

- **PageRank issues**
  - (1) Some nodes are dead ends without out links
    - Random walker does not have a way to go
    - The importance "leaks out" from such nodes



Dead end

  - (2) There exist spider traps: (all links are within the group)
    - Random walker can not go out of the trap
    - At some point spider traps absorb all the importance

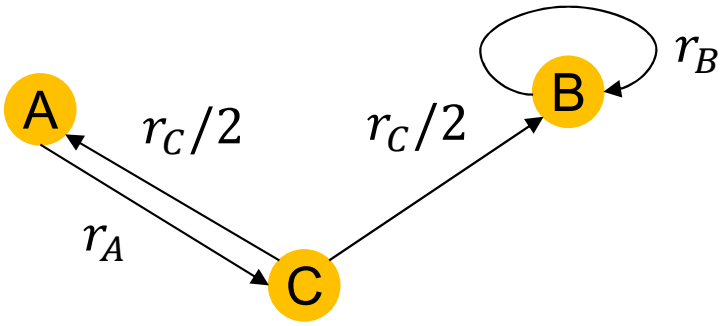


Spider trap

Copyright © 2026 CCL: Analysis of massive data sets

Google Formulation

- **Issue (2): Spider traps**
  - Power Iteration



	A	B	C
A	0	0	$1/2$
B	0	1	$1/2$
C	1	0	0

M

  - Results:

$$\begin{bmatrix} r_A \\ r_B \\ r_C \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \quad \begin{bmatrix} 1/6 \\ 3/6 \\ 2/6 \end{bmatrix} \quad \begin{bmatrix} 1/6 \\ 4/6 \\ 1/6 \end{bmatrix} \quad \dots \quad \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

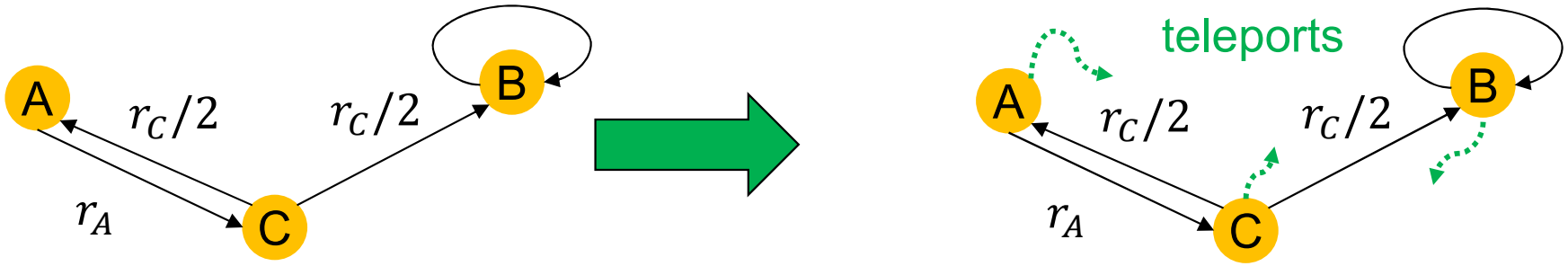
Iteration 0 1 2 …

B is a spider trap; all the importance gets trapped in node B

Copyright © 2026 CCL: Analysis of massive data sets

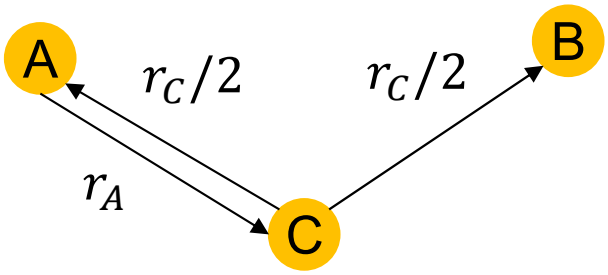
Google Formulation

- **Issue (2): Spider traps**
- **Solution: Teleports!**
  - Google solution:
    At each time step, the random walker has two options:
    1) Follow the random link with some probability β
    2) Jump to some random node with probability $1 - \beta$
  - Common values for β
    - In the range 0.8 to 0.9
    - Random walker will teleport itself out of the spider trap



Google Formulation

- **Issue (1): Dead ends**
  - Power Iteration



	A	B	C
A	0	0	$1/2$
B	0	0	$1/2$
C	1	0	0

M

  - Results:

$$\begin{bmatrix} r_A \\ r_B \\ r_C \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \quad \begin{bmatrix} 1/6 \\ 1/6 \\ 2/6 \end{bmatrix} \quad \begin{bmatrix} 2/12 \\ 1/12 \\ 2/12 \end{bmatrix} \quad \dots \quad \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

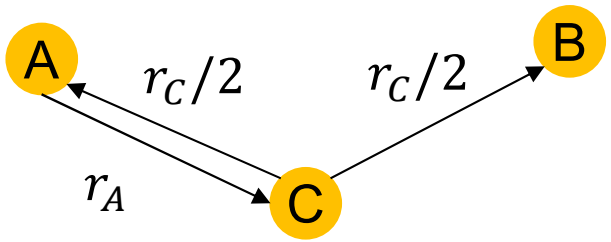
Iteration 0 1 2 …

B is a dead end, all the importance “leaks out” since the matrix is not column-stochastic

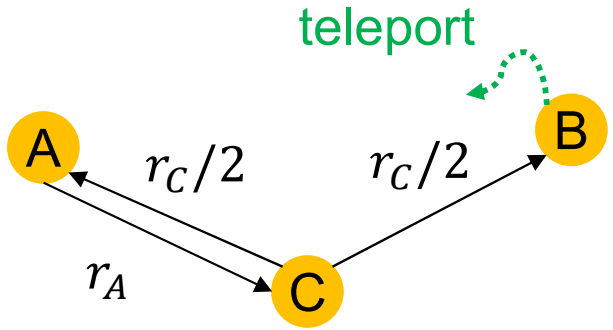
Copyright © 2026 CCL: Analysis of massive data sets

Google Formulation

- **Issue (1): Dead ends**
- **Solution: Teleports!**
  - Google solution:
    Apply random teleports with the probability 1.0 from each dead-end node



	A	B	C
A	0	0	1/2
B	0	0	1/2
C	1	0	0



	A	B	C
A	0	1/3	1/2
B	0	1/3	1/2
C	1	1/3	0

Copyright © 2026 CCL: Analysis of massive data sets

Google Formulation

- **Issues Specifics**
  - Spider traps
    - Spider traps are not the problem for the algorithm in a sense that it will still converge
    - The algorithm will work, but the rank scores are not what we expect
    - Solution: The walker always has the teleporting option, so she will never get stuck in a spider trap
  - Dead ends
    - Dead ends are a problem for the algorithm
    - The assumption was that the matrix is column-stochastic, so with dead ends the initial conditions are not met
    - Solution: For each dead-end node, the walker always follows teleports

Copyright © 2026 CCL: Analysis of massive data sets

Google Formulation

- **PageRank Equation [Brin-Page, '98]**

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + \frac{(1 - \beta)}{N}$$

  - With probability β, the walker follows link
  - With probability $1 - \beta$, the walker teleports to random page
  - This formula assumes that M has no dead ends

- **The Google Matrix A:**

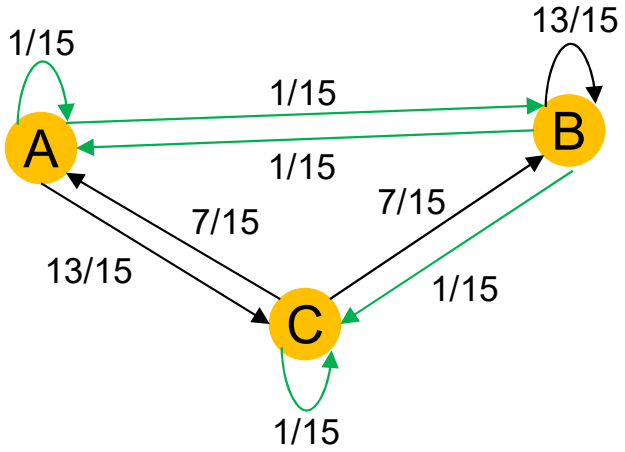
$$A = \beta M + (1 - \beta)\left[\frac{1}{N}\right]_{N \times N}$$

- **Recursive problem: $r = A \cdot r$**
  - The power iteration method works
    - $\beta = 0.8, 0.9$ (follow 5 steps and then jump)

Copyright © 2026 CCL: Analysis of massive data sets

Google Formulation

- **Back to our Example ($\beta = 0.8$)…**



$$A = 0.8 \underbrace{\begin{bmatrix} 0 & 0 & 1/2 \\ 0 & 1 & 1/2 \\ 1 & 0 & 0 \end{bmatrix}}_{M} + 0.2 \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}_{N \times N}$$

$$A = \begin{matrix} A \\ B \\ C \end{matrix} \begin{bmatrix} 1/15 & 1/15 & 7/15 \\ 1/15 & 13/15 & 7/15 \\ 13/15 & 1/15 & 1/15 \end{bmatrix}$$

- **Results:**

$$\begin{bmatrix} r_A \\ r_B \\ r_C \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \quad \begin{bmatrix} 0.20 \\ 0.46 \\ 0.33 \end{bmatrix} \quad \begin{bmatrix} 0.20 \\ 0.58 \\ 0.23 \end{bmatrix} \quad \dots \quad \begin{bmatrix} 7/51 \\ 35/51 \\ 9/51 \end{bmatrix}$$

Iteration 0 1 2 …

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Computing PageRank**
  - Iterative algorithm
    - Uses matrix-vector multiplication
    - $r^{(t+1)} = A \cdot r^{(t)}$
  - It is straightforward if we have enough memory to keep A, $r^{(t)}$ and $r^{(t+1)}$
  - For example, $N = 10^9$ nodes in the graph
    - For each entry we need 4 bytes
    - For 2 vectors, we need 2 billion entries ~ 8GB, it might fit in RAM
    - Matrix A has N^2 entries – 10^{18}

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Matrix Reformulation**
  - Matrix M content
    - Consider node i, with d_i outgoing links
      - $M_{ji} = 1/|d_i|$, if $i \rightarrow j$
      - $M_{ji} = 0$ otherwise
    - Suppose we have 10 links per node, M is very sparse
  - Random teleports make the matrix A dense, but…
    - Adding a teleport link from node i to every node with the probability $(1-\beta)/N$
    - Reducing the probability of following a link from $1/|d_i|$ to $\beta/|d_i|$
    - Equivalent: Tax the importance of each node by $(1-\beta)$ and redistribute it evenly!

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Matrix Reformulation**
  - $r = A \cdot r,\ A_{ji} = \beta M_{ji} + \frac{1-\beta}{N}$
  - $r_j = \sum_{i=1}^{N} A_{ji} \cdot r_i$
  - $r_j = \sum_{i=1}^{N} [\beta M_{ji} + \frac{1-\beta}{N}] \cdot r_i$
  - $r_j = \sum_{i=1}^{N} \beta M_{ji} \cdot r_i + \frac{1-\beta}{N} \sum_{i=1}^{N} r_i$, note that: $\sum_{i=1}^{N} r_i = 1$
  - $r_j = \sum_{i=1}^{N} \beta M_{ji} \cdot r_i + \frac{1-\beta}{N}$

    We assume that **M** has no dead ends
  - Hence, we get

$$r = \beta \cdot M \cdot r + \left[\frac{1-\beta}{N}\right]_N$$

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **New PageRank equation:**

$$r = \beta \cdot M \cdot r + \left[\frac{1-\beta}{N}\right]_N$$

- ***M* is a sparse matrix!**
  - 10 links per node, ~10*N entries
  - In each iteration:
    - Compute $r^{(t+1)} = \beta \mathrm{M} \cdot \mathrm{r}^{(t)}$
    - Add a constant value $(1-\beta)/N$ to each component in $r^{(t+1)}$
    - If M has dead ends
      - $\sum_{i=1}^{N} r_i < 1$, the importance leaks out
      - We need to compute the importance that has leaked out and redistribute it evenly so r still sums to 1, $\sum_{i=1}^{N} r_i = 1$

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

PageRank Algorithm

Input: Graph G (with spider traps and dead ends), β

Output: PageRank vector r

$r_j^{(t)} := 1/N$

do

$S := 0$ //accumulator for non-leaked importance

for $j := 1$ **to** N

$r'^{(t+1)}_j = \sum_{i \to j} \beta \left(\frac{r_i^{(t)}}{d_i}\right)$

$S \mathrel{+}= r'^{(t+1)}_j$

for $j := 1$ **to** N // re-insert the leaked importance

$r'^{(t+1)}_j \mathrel{+}= (1 - S)/N$

while $\left(\sum_j \left| r'^{(t+1)}_j - r_j^{(t)} \right| > \epsilon\right)$

Implementation Details

- **Data Structures**
  - Use adjacency list for graph/matrix representation
    - Encode it using nonzero entries
    - It requires space proportional to the number of links
    - Let's assume web graph
      - $N = 10^9$, 1 billion nodes
      - Space ~ $10 \cdot N = 4 \cdot 10 \cdot 10^9 =$ 40 GB
      - It will not fit in main memory, but it can fit on disk

Source node	Node Degree	Destination nodes
0	2	1, 4
1	6	0, 3, 5, 12, 98, 314
…	…	…
N	5	18, 2, 3, 8, 10

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **PageRank Algorithm**
- **Hard Cases**
  - Case (1):
    - Assume M cannot fit in memory
    - Assume $r^{(t+1)}$ can fit in RAM
  - Case (2):
    - Assume M cannot fit in memory
    - Assume even $r^{(t+1)}$ cannot fit in RAM

Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

Case (1)

- We assume M cannot fit in memory and $r^{(t+1)}$ can fit in RAM
- 1 step of iteration *M and $r^{(t)}$ stored on disk*:

$r^{(t+1)} := (1-\beta)/N$

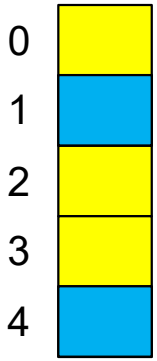
for each node i **with out degree** d_i

  read in memory: $i, d_i, dest_1, dest_2, \ldots, dest_{d_i}, r^{(t)}[i]$

  for $j := 1$ **to** d_i

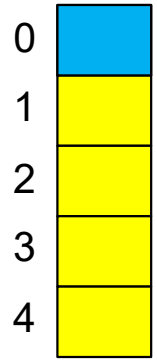
    $r^{(t+1)}[dest_j] \mathrel{+}= \beta\, r^{(t)}[i]/d_i$

$r^{(t+1)}$



Source	Degree	Destination
0	2	1, 4
1	3	0, 2, 5
2	5	18, 2, 3, 8, 10

$r^{(t)}$



Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

Case (1)

- We assume M cannot fit in memory and $r^{(t+1)}$ can fit in RAM
- 1 step of iteration *M and $r^{(t)}$ stored on disk*:

$r^{(t+1)} := (1-\beta)/N$

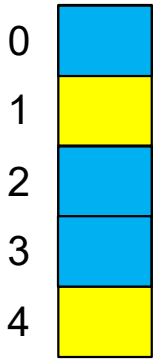
for each node i **with out degree** d_i

  read in memory: $i, d_i, dest_1, dest_2, \ldots, dest_{d_i}, r^{(t)}[i]$

  for $j := 1$ **to** d_i

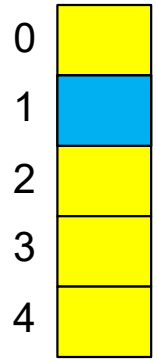
    $r^{(t+1)}[dest_j] \mathrel{+}= \beta\, r^{(t)}[i]/d_i$

$r^{(t+1)}$



Source	Degree	Destination
0	2	1, 4
1	3	0, 2, 3
2	5	18, 2, 3, 8, 10

$r^{(t)}$



Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

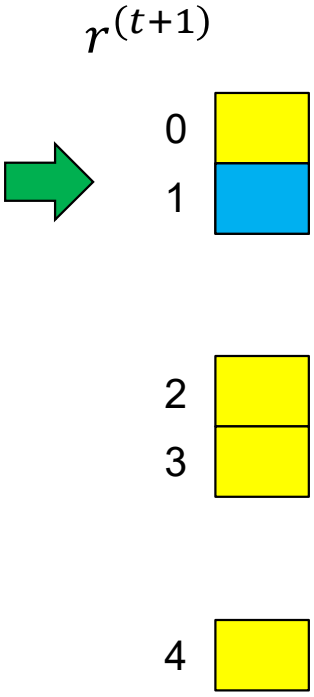
- **Case (1)**
  - Computational Analysis
    - Store $r^{(t)}$ and matrix M on disk
    - In each iteration:
      - Read $r^{(t)}$
      - Write $r^{(t+1)}$
    - Hence, cost per iteration
      - $\boldsymbol{2|r| + |M|}$
  - What should we do in Case (2)?
    - $r^{(t+1)}$ cannot fit in RAM

Copyright © 2026 CCL: Analysis of massive data sets

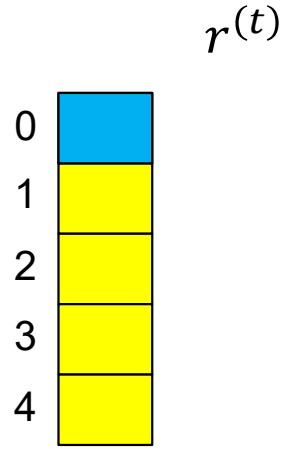
Implementation Details

Case (2)

- We assume neither M nor $r^{(t+1)}$ can fit in RAM
- Solution: we split vector $r^{(t+1)}$ into k blocks that do fit in RAM



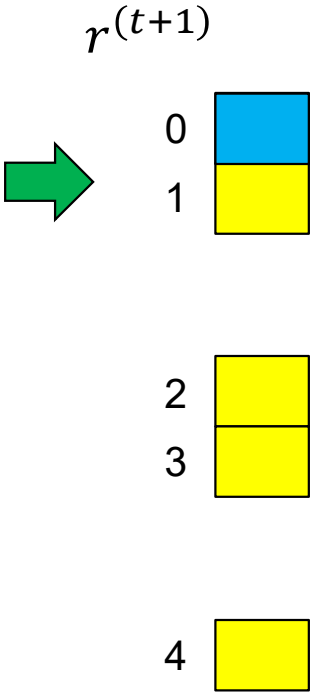
Source	Degree	Destination
0	2	1, 4
1	3	0, 2, 3
2	5	18, 21, 9, 8, 10



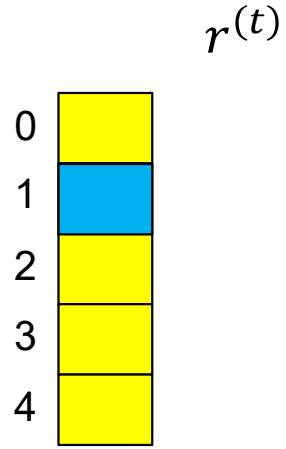
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Case (2)**
  - We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - Solution: we split vector $r^{(t+1)}$ into k blocks that do fit in RAM



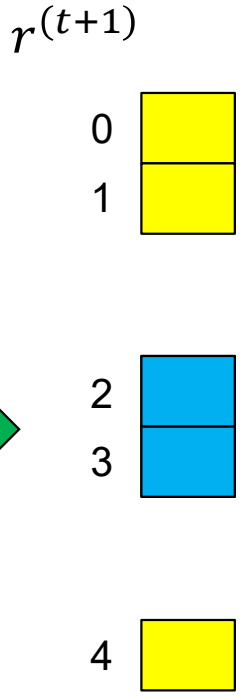
Source	Degree	Destination
0	2	1, 4
1	3	0, 2, 3
2	5	18, 21, 9, 8, 10



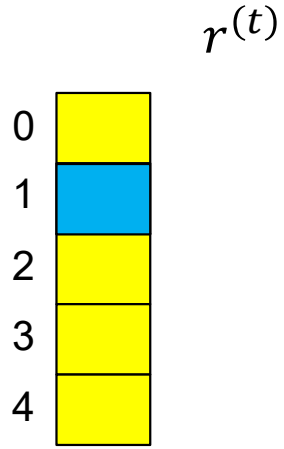
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Case (2)**
  - We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - Solution: we split vector $r^{(t+1)}$ into k blocks that do fit in RAM



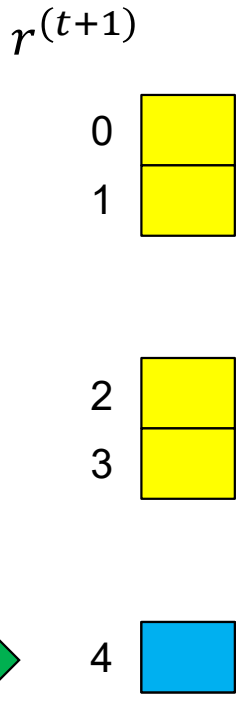
Source	Degree	Destination
0	2	1, 4
1	3	0, 2, 3
2	5	18, 21, 9, 8, 10



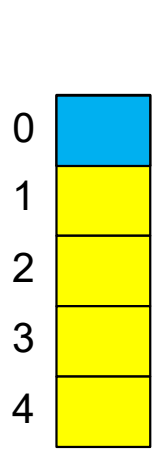
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Case (2)**
  - We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - Solution: we split vector $r^{(t+1)}$ into k blocks that do fit in RAM



Source	Degree	Destination
0	2	1, 4
1	3	0, 2, 3
2	5	18, 21, 9, 8, 10



Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

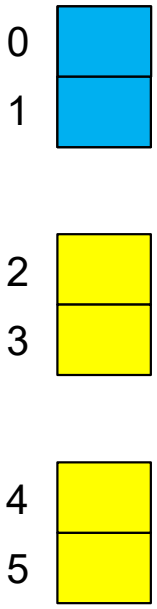
- **Case (2)**
  - Computational Analysis
    - We assume neither M nor $r^{(t+1)}$ can fit in RAM
    - Solution: we split vector $r^{(t+1)}$ into k blocks that do fit in RAM
    - Scan M and $r^{(t)}$ once for each block k_i
  - Cost per iteration
    - $\boldsymbol{k(|M| + |r|)}$
    - M is larger than r, $\sim(10-20)\times$ larger
    - Can we do better?

Implementation Details

Case (2) – Can we do better?

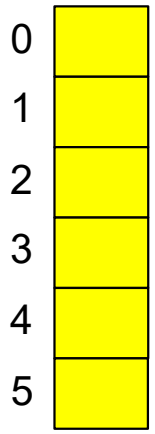
- We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
  - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$

$r^{(t+1)}$



Source	Degree	Destination
0	4	0, 1, 3, 4
1	2	0, 4
2	3	1, 2, 5

$r^{(t)}$

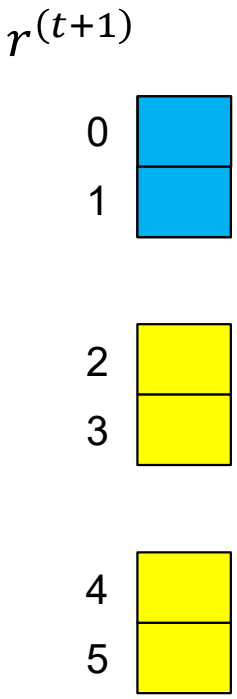


Copyright © 2026 CCL: Analysis of massive data sets

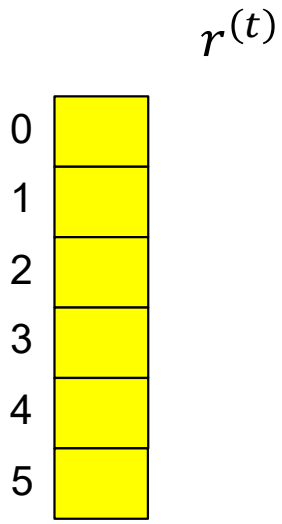
Implementation Details

Case (2) – Can we do better?

- We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
  - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$



Source	Degree	Destination
0	4	0, 1
1	2	0
2	3	1



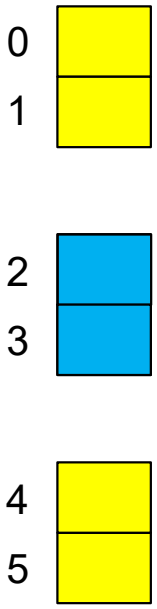
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

Case (2) – Can we do better?

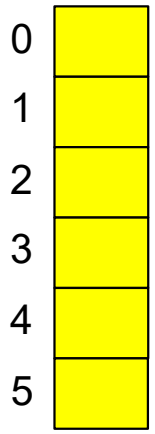
- We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
  - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$

$r^{(t+1)}$



Source	Degree	Destination
0	4	0, 1, 3, 4
1	2	0, 4
2	3	1, 2, 5

$r^{(t)}$



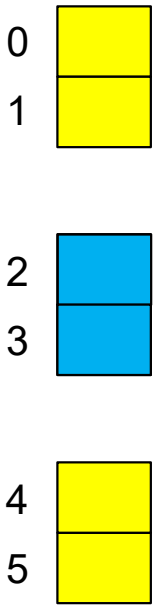
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

Case (2) – Can we do better?

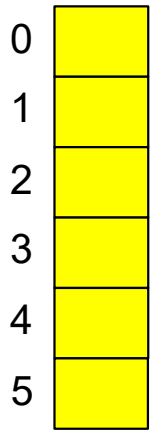
- We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
  - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$

$r^{(t+1)}$



Source	Degree	Destination
0	4	0, 1
1	2	0
2	3	1
0	4	3
2	3	2

$r^{(t)}$



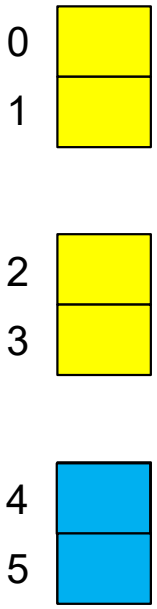
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

Case (2) – Can we do better?

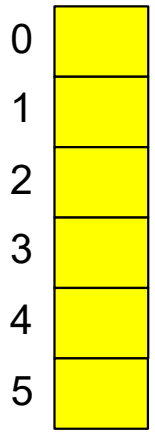
- We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
  - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$

$r^{(t+1)}$



Source	Degree	Destination
0	4	0, 1, 3, 4
1	2	0, 4
2	3	1, 2, 5

$r^{(t)}$



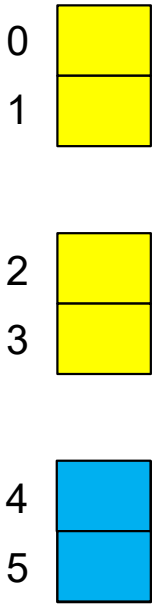
Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

Case (2) – Can we do better?

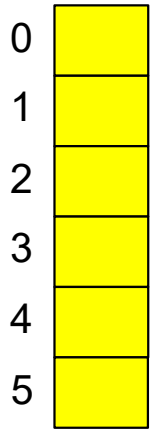
- We assume neither M nor $r^{(t+1)}$ can fit in RAM
  - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
  - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$

$r^{(t+1)}$



Source	Degree	Destination
0	4	0, 1
1	2	0
2	3	1
0	4	3
2	3	2
0	4	4
1	2	4
2	3	5

$r^{(t)}$



Copyright © 2026 CCL: Analysis of massive data sets

Implementation Details

- **Case (2) – Better Approach**
  - Computational Analysis
    - We assume neither M nor $r^{(t+1)}$ can fit in RAM
      - (A) Split vector $r^{(t+1)}$ into k blocks that do fit in RAM
      - (B) Break M into k stripes so each stripe contains only destinations that are in the corresponding block of $r^{(t+1)}$
  - Cost per iteration
    - Some additional overhead per stripe
    - $\boldsymbol{|M|(1+\epsilon) + k|r|}$

PageRank Issues

- **PageRank Issues**
  - It measures general popularity of the node
    - Some topics may not be so popular
    - Solution: topic-specific PageRank algorithm
  - It uses a single measure of importance
    - There are other models of importance
    - Solution: Hubs and Authorities
  - Vulnerable to Link spam
    - One could create artificial structure of nodes in order to boost the importance of particular node
    - Solution: TrustRank

Copyright © 2026 CCL: Analysis of massive data sets

State-of-the-art

- ***PageRank Bandits for Link Prediction***
  - Reframes link prediction as a sequential decision-making problem instead of static prediction
  - https://arxiv.org/abs/2411.01410
- ***Heuristic Learning with Graph Neural Networks***
  - Learn the heuristic instead of designing it manually
  - https://arxiv.org/abs/2406.07979

Copyright © 2026 CCL: Analysis of massive data sets

State-of-the-art

- ***Attribute-Enhanced Similarity Ranking for Sparse Link PredictionLink***
  - Simple approach, challenges the dominance of GNNs.
  - https://arxiv.org/abs/2412.00261
- ***Link Prediction via Language Models (LPNL)***
  - Treats graph reasoning as a language problem
  - https://arxiv.org/pdf/2401.13227